

Options for addressing *requires*-clause syntax ambiguities

Document number: P0730R0
Date: 2017-07-11
Project: ISO/IEC JTC 1/SC 22/WG 21/C++
Working Group: Evolution
Revises: None
Reply-to: Hubert S.K. Tong <hubert.reinterpretcast@gmail.com>

Description from Concepts Issue 21

Given either

```
template <typename T> requires (bool)&T::operator short  
unsigned int foo();
```

or

```
template <typename T> requires (bool)sizeof new (T::f()) short  
unsigned int bar();
```

there is more than one successful parse and it is unclear whether the return type is `unsigned int` or `int`.

The after-the-function-declarator form of the *requires*-clause is also ambiguous:

```
struct X {};  
template<typename T> void f() requires (bool)sizeof new X  
{  
    // might be the function body, might be a brace-or-equal-init for X  
};
```

A “max munch” rule would probably do the wrong thing for the above case, and likewise here:

```
template<typename T> requires (bool)sizeof new unsigned  
struct X { };
```

Proposed resolutions have been to:

- require parens,
- specify rules to make this unambiguous,
- specify the operand of a *requires*-clause as a new grammar.

Current implementation behaviour

Both GCC and Clang currently handle these cases by consuming the maximum sequence of tokens that parse as a *constraint-expression*.

Implementations can generate reasonably good diagnostics for experts that are aware of the issue; however, for users encountering this issue for the first time, this is an unfriendly surprise.

Example (credit due to Richard Smith):

```
template<typename U>
struct A {
    template<typename T = U> requires (bool)(T())
    A() {}
};
```

Diagnostic output from Clang:

```
<stdin>:4:7: error: expected member name or ';' after declaration specifiers
    A() {}
      ^
1 error generated.
```

The above error message indicates that the parsing reached the brace without encountering a *declarator-id*; however, it references “declaration specifiers”, of which none are actually present.

If we standardize on the existing behaviour, then adding parentheses around the intended constraint can be harmful; the *requires-clause* may then appear as the beginning of a C-style cast:

```
template <typename T>
requires (bool(T()))
int (*fp)();
```

Diagnostic output from Clang:

```
<stdin>:3:7: error: use of undeclared identifier 'fp'
int (*fp)();
    ^
1 error generated.
```

The above error message indicates that the parsing reached the intended *declarator-id*, but *fp* is seen as a name being referred to within an expression.

Alternative: require parentheses as part of a *requires-clause*

This alternative is self-explanatory. The parentheses act as a natural delimiter. This would normally cost one extra character because the opening parenthesis is likely to be placed immediately after the *requires* token:

```
template <typename T>
requires C1<T>
void f(T &&);
```

becomes

```
template <typename T>
requires(C1<T>)
void f(T &&);
```

Alternative: custom expression grammar for *requires-clause*

This alternative allows commonly written forms of *constraint-expression* to appear without parentheses. A sample grammar follows:

```
requires-clause:
    requires constraint-logical-or-expression
```

constraint-primary-expression:
 primary-expression
 qualified-concept-name ()

constraint-logical-and-expression:
 constraint-primary-expression
 constraint-logical-and-expression && *constraint-primary-expression*

constraint-logical-or-expression:
 constraint-logical-and-expression
 constraint-logical-or-expression || *constraint-logical-and-expression*

This grammar clearly allows conjunctions and disjunctions to be expressed easily, and leaves invocation of function concepts and *primary-expressions* as “primitives”.

Restricting to the *primary-expression* grammar limits the impact of the issue.

primary-expression:
 literal
 this
 (*expression*)
 id-expression
 lambda-expression
 fold-expression
 requires-expression

We note that *id-expression* is still present and leaves problems with *conversion-type-ids*; however, in such cases, the constraint expression would be broken anyway since it would not have type `bool`.

Constraint primitives not requiring parentheses using this custom grammar include:

- boolean literals, e.g., `true`;
- references to variable concepts, e.g., `C0<T>`;
- references to type traits, e.g., `std::is_union<T>::value` or `std::is_union_v<T>`;
- *fold-expressions*; and
- *requires-expressions*.

Acknowledgements

The author would like to thank Ville Voutilainen and Richard Smith for their input in discussions leading up to the production of this paper. As usual, any mistakes are the responsibility of the author.